

Document number: P1322R3
Date: 2021-02-05
Project: Programming Language C++
Audience: SG4 - Networking, LEWG
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

Networking TS enhancement to enable custom I/O executors

1. Introduction

At present, the Networking TS's I/O objects -- sockets, acceptors, resolvers, and timers -- can be associated only with concrete execution contexts of type `io_context`. This paper proposes a minor change to the specification to permit the construction of these objects with arbitrary I/O executors and execution contexts. This would:

- allow implementers to support multiple "native" execution contexts, with different design and performance trade-offs; and
- simplify some use cases, by allowing users to specify their preferred executor once, at construction, rather than on each operation.

2. Rationale

Construction with arbitrary execution contexts

Users and third-party libraries may implement their own execution contexts, for example to provide different scheduling guarantees, or to natively support other types of I/O objects, such as files. This proposal would allow users to create the standard I/O objects, such as sockets, to be associated with these custom execution contexts. However, although constructing the socket with a custom execution context will provide the correct, expected behaviour, there is no guarantee that it will operate as efficiently as a socket associated with a native execution context.

Support for multiple native execution contexts

Some operating systems have multiple potential implementation strategies for the I/O objects, like sockets. For example, on Windows we can choose to have asynchronous notifications delivered via a completion port to a user-created thread, or we can have them delivered directly to the system thread pool. An implementer may want to make both of these strategies available, via the `io_context` and `system_context` execution contexts, respectively. For example:

```
io_context my_context;  
tcp::socket my_socket_1(my_context); // Implementation uses a completion port.  
  
// ...  
  
tcp::socket my_socket_2(system_executor{}); // Implementation uses the system thread pool.
```

Libraries can also provide additional native execution contexts as implementation-specific extensions.

Convenient construction with a preferred executor

In some use cases, all asynchronous operations on a given I/O object are performed using the same executor. One common example of this is when using a strand. This proposal allows this executor to be specified once, when the I/O object is constructed. For example:

```
io_context my_context;  
  
// ...  
  
strand<io_context::executor_type> my_strand(my_context.get_executor());  
tcp::socket my_socket(my_strand);  
  
// ...  
  
my_socket.async_receive(my_buffer, my_handler); // my_handler is invoked in the strand
```

Vendor- or user-specialization of I/O object templates

The Executor template parameter also offers an opportunity to specialize the I/O object types for specific executor types. For example, some platforms support vendor-specific kernel bypass APIs for I/O such as sockets, enabling significantly lower latency and higher throughput when compared to system calls. Furthermore, these APIs are often limited to supporting only TCP and UDP. A library or user may therefore choose to define an executor type that represents this API as a venue for I/O execution:

```
class vendor_xyz_kernel_bypass_executor  
{
```

```
// ...
};
```

and specialize the socket templates for this executor type:

```
template<>
class basic_socket<ip::tcp, vendor_xyz_kernel_bypass_executor>
{
    // ...
};

template<>
class basic_datagram_socket<ip::tcp, vendor_xyz_kernel_bypass_executor>
    : public basic_socket<ip::tcp, vendor_xyz_kernel_bypass_executor>
{
    // ...
    template<class ConstBufferSequence>
    void send(const ConstBufferSequence& buffers)
    {
        // Implemented using vendor_xyz_send().
        // ...
    }
    // ...
};
```

3. Overview of changes to specification

Addition of Executor template parameter

A new Executor template parameter is added to the following templates:

- basic_waitable_timer
- basic_socket
- basic_datagram_socket
- basic_stream_socket
- basic_socket_acceptor
- basic_socket_streambuf
- basic_socket_iostream
- ip::basic_resolver

To enable convenient interoperability with arbitrary executors and execution contexts, the template parameter is defaulted to the executor polymorphic wrapper. For example:

```
template <class Protocol, class Executor = executor>
    class basic_socket;
```

For each of:

- basic_waitable_timer
- basic_socket
- basic_socket_acceptor
- ip::basic_resolver

the existing nested type executor_type is altered to refer to the Executor type:

```
template <class Protocol, class Executor>
class basic_socket : public socket_base
{
public:
    using executor_type = io_context::executor_typeExecutor;
    // ...
};
```

For the remaining four templates:

- basic_datagram_socket
- basic_stream_socket
- basic_socket_streambuf
- basic_socket_iostream

the nested type executor_type is added:

```
template <class Protocol, class Executor>
class basic_stream_socket : public basic_socket<Protocol, Executor>
```

```
{
public:
    using executor_type = Executor;
    // ...
};
```

Addition of nested template type alias `rebind_executor`

For each of the following templates:

- `basic_waitable_timer`
- `basic_socket`
- `basic_datagram_socket`
- `basic_stream_socket`
- `basic_socket_acceptor`
- `ip::basic_resolver`

a nested template type alias `rebind_executor` is added:

```
template<class Protocol, class Executor>
class basic_socket : public socket_base
{
    // ...

    /// The socket type when rebound to the specified executor.
    template<class OtherExecutor>
    using rebind_executor =
        basic_socket<Protocol, OtherExecutor>;

    // ...
};
```

This enables computation of the appropriate class type for different I/O executor types.

Modification of I/O objects' constructors

For each of the following templates:

- `basic_waitable_timer`
- `basic_socket`
- `basic_datagram_socket`
- `basic_stream_socket`
- `basic_socket_acceptor`
- `ip::basic_resolver`

every constructor with an `io_context&` parameter is replaced with two constructors:

```
template<class Protocol, class Executor>
class basic_socket : public socket_base
{
    // ...
    basic_socket(io_context& ctx, const protocol_type& protocol);
    basic_socket(const executor_type& ex, const protocol_type& protocol);
    template<class ExecutionContext>
    basic_socket(ExecutionContext& ctx, const protocol_type& protocol);
    // ...
};
```

The second of these constructors shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t>::value` is true, and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

Modification of I/O objects' converting constructors

An `OtherExecutor` template parameter is added as required to the converting move constructors of the following classes:

- `basic_socket`
- `basic_datagram_socket`
- `basic_stream_socket`
- `basic_socket_acceptor`

For example:

```
template<class Protocol, class Executor>
class basic_socket : public socket_base
{
    // ...
    template<class OtherProtocol, class OtherExecutor>
        basic_socket(basic_socket<OtherProtocol, OtherExecutor>&& rhs);
    // ...
};
```

This constructor shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol`, and `OtherExecutor` is implicitly convertible to `Executor`.

Modification of I/O objects' converting assignment operators

An `OtherExecutor` template parameter is added as required to the converting move assignment operators of the following classes:

- `basic_socket`
- `basic_datagram_socket`
- `basic_stream_socket`
- `basic_socket_acceptor`

For example:

```
template<class Protocol, class Executor>
class basic_socket : public socket_base
{
    // ...
    template<class OtherProtocol, class OtherExecutor>
        basic_socket& operator=(basic_socket<OtherProtocol, OtherExecutor>&& rhs);
    // ...
};
```

This assignment operator shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol`, and `OtherExecutor` is implicitly convertible to `Executor`.

Addition of nested type alias `socket_for` to `AcceptableProtocol` requirements and protocol classes

The `AcceptableProtocol` type requirements, `ip::tcp` class, and `ip::udp` class each acquire a new nested template type alias `socket_for`. It can be used to create a protocol socket with a specific executor type:

```
net::ip::tcp::socket_for<my_executor> sock = /* ... */;
```

Addition of template type alias to `basic_socket_acceptor` for using a socket with a specific executor

The template `basic_socket_acceptor` gains a new nested template type alias `socket_type_for`:

```
template<class AcceptableProtocol, class Executor>
class basic_socket_acceptor : public socket_base
{
    // ...

    template<class OtherExecutor>
    using socket_type_for = typename AcceptableProtocol::socket_for<OtherExecutor>;

    // ...
};
```

This enables computation of the appropriate socket type produced when accepting on different I/O executor types.

Modification of `basic_socket_acceptor` member functions

Every overload of member functions `accept` and `async_accept` with an `io_context&` parameter is replaced with two overloads:

```
template<class AcceptableProtocol, class Executor>
class basic_socket_acceptor : public socket_base
{
    // ...
    socket_type_for<io_context& ctx> accept(io_context& ctx);
    template<class OtherExecutor>
    socket_type_for<OtherExecutor>
    accept(const OtherExecutor& ex);
    template<class ExecutionContext>
    socket_type_for<typename ExecutionContext::executor_type>
    accept(ExecutionContext& ctx);
};
```

```
// ...  
};
```

The second of these overloads shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true, and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

Modification of `connect` and `async_connect` functions

An Executor template parameter is added as required to the `connect` and `async_connect` functions. For example:

```
template<class Protocol, class Executor, class EndpointSequence>  
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,  
                                       const EndpointSequence& endpoints);
```

Modification of `basic_socket_streambuf`

The `basic_socket_streambuf` default constructor is modified so that it shall not participate in overload resolution unless `is_constructible<executor_type, io_context::executor_type>::value` is true.

Modification of `basic_socket_iostream`

The `basic_socket_iostream` specification is modified so that the `executor_type` type is passed to all uses of `basic_socket_streambuf`, as required. The `basic_socket_iostream` default constructor is modified so that it shall not participate in overload resolution unless `is_default_constructible<basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>>::value` is true.

4. Possible implementation approach

This section is intended to provide implementation suggestions only, and is not intended to be prescriptive or exhaustive.

Provision of default implementation

Implementations may use an execution context service as a container for the backend implementation.

```
class __socket_backend : public execution_context::service  
{  
    // ...  
};
```

This service would be "used" by an I/O object implementation, and is automatically created on first use:

```
template <class Protocol, class Executor>  
class basic_socket : public socket_base  
{  
public:  
    using executor_type = Executor;  
  
    // ...  
  
    explicit basic_socket(const executor_type& ex)  
    {  
        auto& backend = use_service<__socket_backend>(ex.context());  
        // ...  
    }  
  
    // ...  
};
```

This allows the I/O object to be used with arbitrary execution contexts.

Native implementations

A native execution context preemptively performs service creation by calling `make_service`. It can use this opportunity to pass additional constructor arguments that initialise the backend in the native mode, rather than the default:

```
class io_context : public execution_context  
{  
public:  
    // ...  
  
    io_context()
```

```

{
    make_service<__socket_backend>(*this, __io_context_backend_tag{});
}

// ...
};

```

Alternatively, implementations may use a class hierarchy of services, and virtual functions, to select the desired behaviour:

```

class __socket_backend : public execution_context::service
{
public:
    using key_type = __socket_backend;

    // ...

    virtual void backend_function(/* ... */);

    // ...
};

class __io_socket_backend : public __socket_backend
{
public:
    // ...

    void backend_function(/* ... */) override;

    // ...
};

class io_context : public execution_context
{
public:
    // ...

    io_context()
    {
        make_service<__io_socket_backend>(*this);
    }

    // ...
};

```

In both approaches, the existing service, with its native backend, is obtained by the I/O object constructor.

Performance impact

Specification of the polymorphic executor as the default I/O executor, while improving usability, has a non-zero impact on performance. This impact can be mitigated by having an I/O object's constructor detect its own well-known native executor types (e.g. by using `executor::target_type`). With this information, the overhead can then be limited to simple branching, rather than the memory allocations and virtual functions that would likely be required for the type-erased function objects.

Users can avoid this overhead completely by explicitly specifying the I/O executor type:

```

io_context my_context;
basic_stream_socket<tcp, io_context::executor_type> my_socket(my_context);

```

5. Implementation experience

This design change was implemented in Asio 1.13.0, and was delivered in the Boost 1.70 release as the stable Asio version 1.14.0.

6. Proposed changes to wording

These changes are relative to N4771.

Update the `<experimental/net_fwd>` synopsis *[fwd.decl.synop]* as follows:

12.1 Header `<experimental/net_fwd>` synopsis [\[fwd.decl.synop\]](#)

```

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

```

```

class execution_context;
template<class T, class Executor>
    class executor_binder;
template<class Executor>
    class executor_work_guard;
class system_executor;
class executor;
template<class Executor>
    class strand;

class io_context;

template<class Clock> struct wait_traits;
template<class Clock, class WaitTraits = wait_traits<Clock>, class Executor = executor>
    class basic_waitable_timer;
using system_timer = basic_waitable_timer<chrono::system_clock>;
using steady_timer = basic_waitable_timer<chrono::steady_clock>;
using high_resolution_timer = basic_waitable_timer<chrono::high_resolution_clock>;

template<class Protocol, class Executor = executor>
    class basic_socket;
template<class Protocol, class Executor = executor>
    class basic_datagram_socket;
template<class Protocol, class Executor = executor>
    class basic_stream_socket;
template<class Protocol, class Executor = executor>
    class basic_socket_acceptor;
template<class Protocol, class Clock = chrono::steady_clock,
    class WaitTraits = wait_traits<Clock>, class Executor = executor>
    class basic_socket_streambuf;
template<class Protocol, class Clock = chrono::steady_clock,
    class WaitTraits = wait_traits<Clock>, class Executor = executor>
    class basic_socket_iostream;

namespace ip {

    class address;
    class address_v4;
    class address_v6;
    template<class Address>
        class basic_address_iterator;
    using address_v4_iterator = basic_address_iterator<address_v4>;
    using address_v6_iterator = basic_address_iterator<address_v6>;
    template<class Address>
        class basic_address_range;
    using address_v4_range = basic_address_range<address_v4>;
    using address_v6_range = basic_address_range<address_v6>;
    class network_v4;
    class network_v6;
    template<class InternetProtocol>
        class basic_endpoint;
    template<class InternetProtocol>
        class basic_resolver_entry;
    template<class InternetProtocol>
        class basic_resolver_results;
    template<class InternetProtocol, class Executor = executor>
        class basic_resolver;
    class tcp;
    class udp;

} // namespace ip
} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

Update the `<experimental/timer>` synopsis [timer.synop] as follows:

15.1 Header `<experimental/timer>` synopsis

[\[timer.synop\]](#)

[...]

```
template<class Clock, class WaitTraits = wait_traits<Clock>,class Executor = executor>
class basic_waitable_timer;
```

Update the class template `basic_waitable_timer` [`timer.waitable`] as follows:

15.4 Class template `basic_waitable_timer` [`timer.waitable`]

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

template<class Clock, class WaitTraits = wait_traits<Clock>,class Executor = executor>
class basic_waitable_timer
{
public:
// types:

using executor_type = io_context::executor_typeExecutor;
using clock_type = Clock;
using duration = typename clock_type::duration;
using time_point = typename clock_type::time_point;
using traits_type = WaitTraits;

template<class OtherExecutor>
using rebind_executor =
basic_waitable_timer<Clock, WaitTraits, OtherExecutor>;

// [timer.waitable.cons], construct / copy / destroy:

explicit basic_waitable_timer(io_context& ctxconst executor_type& ex);
template<class ExecutionContext>
explicit basic_waitable_timer(ExecutionContext& ctx);
basic_waitable_timer(io_context& ctxconst executor_type& ex, const time_point& t);
template<class ExecutionContext>
basic_waitable_timer(ExecutionContext& ctx, const time_point& t);
basic_waitable_timer(io_context& ctxconst executor_type& ex, const duration& d);
template<class ExecutionContext>
basic_waitable_timer(ExecutionContext& ctx, const duration& d);
basic_waitable_timer(const basic_waitable_timer&) = delete;
basic_waitable_timer(basic_waitable_timer&& rhs);
```

Update the `basic_waitable_timer` constructors [`timer.waitable.cons`] as follows:

15.4.1 `basic_waitable_timer` constructors [`timer.waitable.cons`]

```
explicit basic_waitable_timer(io_context& ctxconst executor_type& ex);
```

1 *Effects:* Equivalent to `basic_waitable_timer(ctxex, time_point())`.

```
template<class ExecutionContext>
explicit basic_waitable_timer(ExecutionContext& ctx);
```

Effects: Equivalent to `basic_waitable_timer(ctx.get_executor())`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t>::value` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is `true`.

```
basic_waitable_timer(io_context& ctxconst executor_type& ex, const time_point& t);
```

2 *Postconditions:*

(2.1) — `get_executor() == ctxex.get_executor()`.

(2.2) — `expiry() == t`.

```
template<class ExecutionContext>
basic_waitable_timer(ExecutionContext& ctx, const time_point& t);
```

Effects: Equivalent to `basic_waitable_timer(ctx.get_executor(), t)`.

Remarks:This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution context&>::value` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is `true`.

`basic_waitable_timer(io_context& ctxconst executor_type& ex, const duration& d);`

3 *Effects:*Sets the expiry time as if by calling `expires_after(d)`.

4 *Postconditions:*`get_executor() == ctx.get_executor()ex`.

`template<class ExecutionContext>`

`basic_waitable_timer(ExecutionContext& ctx, const duration& d);`

*Effects:*Equivalent to `basic_waitable_timer(ctx.get_executor(), d)`.

Remarks:This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution context&>::value` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is `true`.

Update the `<experimental/socket>` synopsis [`socket.synop`] as follows:

18.1 Header `<experimental/socket>` synopsis [`socket.synop`]

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

enum class socket_errc {
    already_open = an implementation-defined non-zero value,
    not_found = an implementation-defined non-zero value
};

const error_category& socket_category() noexcept;

error_code make_error_code(socket_errc e) noexcept;
error_condition make_error_condition(socket_errc e) noexcept;

// Sockets:

class socket_base;

template<class Protocol, class Executor = executor>
class basic_socket;

template<class Protocol, class Executor = executor>
class basic_datagram_socket;

template<class Protocol, class Executor = executor>
class basic_stream_socket;

template<class Protocol, class Executor = executor>
class basic_socket_acceptor;

// [socket.iostreams], Socket streams:

template<class Protocol, class Clock = chrono::steady_clock,
class WaitTraits = wait_traits<Clock>, class Executor = executor>
class basic_socket_streambuf;

template<class Protocol, class Clock = chrono::steady_clock,
class WaitTraits = wait_traits<Clock>, class Executor = executor>
class basic_socket_iostream;

// [socket.algo.connect], synchronous connect operations:

template<class Protocol, class Executor, class EndpointSequence>
typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
const EndpointSequence& endpoints);

template<class Protocol, class Executor, class EndpointSequence>
typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
const EndpointSequence& endpoints,
```

```

        error_code& ec);
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        ConnectCondition c);
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        ConnectCondition c,
        error_code& ec);

template<class Protocol, class Executor, class InputIterator>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last);
template<class Protocol, class Executor, class InputIterator>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        error_code& ec);
template<class Protocol, class Executor, class InputIterator, class ConnectCondition>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        ConnectCondition c);
template<class Protocol, class Executor, class InputIterator, class ConnectCondition>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        ConnectCondition c,
        error_code& ec);

// [socket.algo.async.connect], asynchronous connect operations:

template<class Protocol, class Executor, class EndpointSequence, class CompletionToken>
    DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        CompletionToken&& token);
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition,
    class CompletionToken>
    DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        ConnectCondition c,
        CompletionToken&& token);

template<class Protocol, class Executor, class InputIterator, class CompletionToken>
    DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        CompletionToken&& token);
template<class Protocol, class Executor, class InputIterator, class ConnectCondition,
    class CompletionToken>
    DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        ConnectCondition c,
        CompletionToken&& token);

} // inline namespace v1
} // namespace net
} // namespace experimental

template<> struct is_error_code_enum<experimental::net::v1::socket_errc>
    : public true_type {};

} // namespace std

```

Update the `AcceptableProtocol` requirements [socket.reqmts.acceptableprotocol] as follows:

18.2.7 Acceptable protocol requirements [socket.reqmts.acceptableprotocol]

- ¹ A type `X` meets the `AcceptableProtocol` requirements if it satisfies the requirements of `Protocol` ([socket.reqmts.protocol]) as well as the additional requirements listed below.

In the table below, `E` is a type that satisfies the `Executor` requirements ([async.reqmts.executor]).

Table 26: `AcceptableProtocol` requirements

expression	return type	assertion/note pre/post-conditions
X::socket	A type that satisfies the requirements of Destructible (C++2014[destructible]) and MoveConstructible (C++2014[moveconstructible]), and that is publicly and unambiguously derived from basic_socket<X>.	
<u>X:: socket_for<E></u>	<u>A type that satisfies the requirements of Destructible (C++2014[destructible]) and MoveConstructible (C++2014[moveconstructible]), and that is publicly and unambiguously derived from basic_socket<X, E>.</u>	

Update the class socket_base [socket.base] as follows:

18.4

Class socket_base

[socket.base]

```

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

class socket_base
{
public:
class broadcast;
class debug;
class do_not_route;
class keep_alive;
class linger;
class out_of_band_inline;
class receive_buffer_size;
class receive_low_watermark;
class reuse_address;
class send_buffer_size;
class send_low_watermark;

using shutdown_type = T1;
static constexpr shutdown_type shutdown_receive;
static constexpr shutdown_type shutdown_send;
static constexpr shutdown_type shutdown_both;

using wait_type = T2;
static constexpr wait_type wait_read;
static constexpr wait_type wait_write;
static constexpr wait_type wait_error;

using message_flags = T3;
static constexpr message_flags message_peek;
static constexpr message_flags message_out_of_band;
static constexpr message_flags message_do_not_route;

static const int max_listen_connections;

protected:
socket_base();
~socket_base();
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

¹ socket_base defines several member types:

- (1.1) — socket option classes broadcast, debug, do_not_route, keep_alive, linger, out_of_band_inline, receive_buffer_size, receive_low_watermark, reuse_address, send_buffer_size, and send_low_watermark;
- (1.2) —

an enumerated type, shutdown_type, for use with the basic_socket<Protocol, [Executor](#)> class's shutdown member function.

- (1.3) — an enumerated type, wait_type, for use with the basic_socket<Protocol, [Executor](#)> and basic_socket_acceptor<Protocol, [Executor](#)> classes' wait and async_wait member functions,
- (1.4) — a bitmask type, message_flags, for use with the basic_stream_socket<Protocol, [Executor](#)> class's send, async_send, receive, and async_receive member functions, and the basic_datagram_socket<Protocol> class's send, async_send, send_to, async_send_to, receive, async_receive, receive_from, and async_receive_from member functions.
- (1.5) — a constant, max_listen_connections, for use with the basic_socket_acceptor<Protocol, [Executor](#)> class's listen member function.

Update the class template basic_socket [socket.basic] as follows:

18.6 Class template basic_socket

[\[socket.basic\]](#)

- 1 Class template basic_socket<Protocol, [Executor](#)> is used as the base class for the basic_datagram_socket<Protocol, [Executor](#)> and basic_stream_socket<Protocol, [Executor](#)> class templates. It provides functionality that is common to both types of socket.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

template<class Protocol, class Executor>
class basic_socket : public socket_base
{
public:
    // types:

    using executor_type = io\_context::executor\_typeExecutor;
    using native_handle_type = implementation-defined; // see \[socket.reqmts.native\]
    using protocol_type = Protocol;
    using endpoint_type = typename protocol_type::endpoint;

    template<class OtherExecutor>
    using rebind\_executor =
    basic\_socket<Protocol, OtherExecutor>;

    // \[socket.basic.ops\], basic_socket operations:

    executor_type get_executor() noexcept;

    native_handle_type native_handle(); // see \[socket.reqmts.native\]

    void open(const protocol_type& protocol = protocol_type());
    void open(const protocol_type& protocol, error_code& ec);

    void assign(const protocol_type& protocol,
                const native_handle_type& native_socket); // see \[socket.reqmts.native\]
    void assign(const protocol_type& protocol,
                const native_handle_type& native_socket,
                error_code& ec); // see \[socket.reqmts.native\]

    native_handle_type release(); // see \[socket.reqmts.native\]
    native_handle_type release(error_code& ec); // see \[socket.reqmts.native\]

    bool is_open() const noexcept;

    void close();
    void close(error_code& ec);

    void cancel();
    void cancel(error_code& ec);

    template<class SettableSocketOption>
    void set_option(const SettableSocketOption& option);
```

```

template<class SettableSocketOption>
    void set_option(const SettableSocketOption& option, error_code& ec);

template<class GettableSocketOption>
    void get_option(GettableSocketOption& option) const;
template<class GettableSocketOption>
    void get_option(GettableSocketOption& option, error_code& ec) const;

template<class IoControlCommand>
    void io_control(IoControlCommand& command);
template<class IoControlCommand>
    void io_control(IoControlCommand& command, error_code& ec);

void non_blocking(bool mode);
void non_blocking(bool mode, error_code& ec);
bool non_blocking() const;

void native_non_blocking(bool mode);
void native_non_blocking(bool mode, error_code& ec);
bool native_non_blocking() const;

bool at_mark() const;
bool at_mark(error_code& ec) const;

size_t available() const;
size_t available(error_code& ec) const;

void bind(const endpoint_type& endpoint);
void bind(const endpoint_type& endpoint, error_code& ec);

void shutdown(shutdown_type what);
void shutdown(shutdown_type what, error_code& ec);

endpoint_type local_endpoint() const;
endpoint_type local_endpoint(error_code& ec) const;

endpoint_type remote_endpoint() const;
endpoint_type remote_endpoint(error_code& ec) const;

void connect(const endpoint_type& endpoint);
void connect(const endpoint_type& endpoint, error_code& ec);

template<class CompletionToken>
    DEDUCED async_connect(const endpoint_type& endpoint,
                          CompletionToken&& token);

void wait(wait_type w);
void wait(wait_type w, error_code& ec);

template<class CompletionToken>
    DEDUCED async_wait(wait_type w, CompletionToken&& token);

```

protected:

// [\[socket.basic.cons\]](#), construct / copy / destroy:

```

explicit basic_socket(io_context& ctx const executor_type& ex);
template<class ExecutionContext>
    explicit basic_socket(ExecutionContext& ctx);
basic_socket(io_context& ctx const executor_type& ex, const protocol_type& protocol);
template<class ExecutionContext>
    basic_socket(ExecutionContext& ctx, const protocol_type& protocol);
basic_socket(io_context& ctx const executor_type& ex, const endpoint_type& endpoint);
template<class ExecutionContext>
    basic_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
basic_socket(io_context& ctx const executor_type& ex, const protocol_type& protocol,
            const native_handle_type& native_socket); // see \[socket.reqmts.native\]
template<class ExecutionContext>
    basic_socket(ExecutionContext& ctx, const protocol_type& protocol,
                const native_handle_type& native_socket); // see \[socket.reqmts.native\]
basic_socket(const basic_socket&) = delete;
basic_socket(basic_socket&& rhs);
template<class OtherProtocol, class OtherExecutor>
    basic_socket(basic_socket<OtherProtocol, OtherExecutor>&& rhs);

```

```

~basic_socket();

basic_socket& operator=(const basic_socket&) = delete;
basic_socket& operator=(basic_socket&& rhs);
template<class OtherProtocol, class OtherExecutor>
    basic_socket& operator=(basic_socket<OtherProtocol, OtherExecutor>&& rhs);

private:
    protocol_type protocol_; // exposition only
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

Update the `basic_socket` constructors [socket.basic.cons] as follows:

18.6.1 basic_socket constructors

[socket.basic.cons]

```
explicit basic_socket(io_context& ctxconst executor_type& ex);
```

1 Postconditions:

(1.1) — `get_executor()` == ~~`ctx.get_executor()`~~`ex`.

(1.2) — `is_open()` == `false`.

```
template<class ExecutionContext>
explicit basic_socket(ExecutionContext& ctx);
```

Effects:Equivalent to `basic_socket(ctx.get_executor())`.

Remarks:This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is `true`.

```
basic_socket(io_context& ctxconst executor_type& ex, const protocol_type& protocol);
```

2 *Effects:*Opens this socket as if by calling `open(protocol)`.

3 Postconditions:

(3.1) — `get_executor()` == ~~`ctx.get_executor()`~~`ex`.

(3.2) — `is_open()` == `true`.

(3.3) — `non_blocking()` == `false`.

(3.4) — `protocol_` == `protocol`.

```
template<class ExecutionContext>
explicit basic_socket(ExecutionContext& ctx, const protocol_type& protocol);
```

Effects:Equivalent to `basic_socket(ctx.get_executor(), protocol)`.

Remarks:This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is `true`.

```
basic_socket(io_context& ctxconst executor_type& ex, const endpoint_type& endpoint);
```

4 *Effects:*Opens and binds this socket as if by calling:

```
open(endpoint.protocol());
bind(endpoint);
```

5 Postconditions:

(5.1) — `get_executor()` == ~~`ctx.get_executor()`~~`ex`.

(5.2) — `is_open()` == `true`.

(5.3) — `non_blocking()` == `false`.

(5.4) — protocol_ == endpoint.protocol();

```
template<class ExecutionContext>  
explicit basic_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
```

Effects:Equivalent to basic_socket(ctx.get_executor(), endpoint).

Remarks:This function shall not participate in overload resolution unless is -
convertible<ExecutionContext&, execution context>::value is true and is -
constructible<executor_type, typename ExecutionContext::executor_type>::value is true.

```
basic_socket(io_context& ctxconst_executor_type& ex, const protocol_type& protocol,  
            const native_handle_type& native_socket);
```

6 *Requires:*native_socket is a native handle to an open socket.

7 *Effects:*Assigns the existing native socket into this socket as if by calling assign(protocol, native_socket).

8 *Postconditions:*

(8.1) — get_executor() == ~~ctx.get_executor()~~ex.

(8.2) — is_open() == true.

(8.3) — non_blocking() == false.

(8.4) — protocol_ == protocol.

```
template<class ExecutionContext>  
basic_socket(ExecutionContext& ctx, const protocol_type& protocol,  
            const native_handle_type& native_socket);
```

Effects:Equivalent to basic_socket(ctx.get_executor(),_protocol_, native_socket).

Remarks:This function shall not participate in overload resolution unless is -
convertible<ExecutionContext&, execution context>::value is true and is -
constructible<executor_type, typename ExecutionContext::executor_type>::value is true.

```
basic_socket(basic_socket&& rhs);
```

9 *Effects:*Move constructs an object of class basic_socket<Protocol, Executor> that refers to the state originally represented by rhs.

10 *Postconditions:*

(10.1) — get_executor() == rhs.get_executor().

(10.2) — is_open() returns the same value as rhs.is_open() prior to the constructor invocation.

(10.3) — non_blocking() returns the same value as rhs.non_blocking() prior to the constructor invocation.

(10.4) — native_handle() returns the prior value of rhs.native_handle().

(10.5) — protocol_ is the prior value of rhs.protocol_.

(10.6) — rhs.is_open() == false.

```
template<class OtherProtocol, class OtherExecutor>  
basic_socket(basic_socket<OtherProtocol, OtherExecutor>&& rhs);
```

11 *Requires:*OtherProtocol is implicitly convertible to Protocol and OtherExecutor is implicitly convertible to Executor.

12 *Effects:*Move constructs an object of class basic_socket<Protocol, Executor> that refers to the state originally represented by rhs.

13 *Postconditions:*

(13.1) — get_executor() == rhs.get_executor().

(13.2) — is_open() returns the same value as rhs.is_open() prior to the constructor invocation.

(13.3) — non_blocking() returns the same value as rhs.non_blocking() prior to the constructor invocation.

(13.4)

- `native_handle()` returns the prior value of `rhs.native_handle()`.
- (13.5) — `protocol_` is the result of converting the prior value of `rhs.protocol_`.
- (13.6) — `rhs.is_open() == false`.
- 14 *Remarks:* This constructor shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

Update the `basic_socket` assignment operators [`socket.basic.assign`] as follows:

18.6.3 `basic_socket` assignment

[[socket.basic.assign](#)]

```
basic_socket& operator=(basic_socket&& rhs);
```

- 1 *Effects:* If `is_open()` is `true`, cancels all outstanding asynchronous operations associated with this socket. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields `true`. Disables the linger socket option to prevent the assignment from blocking, and releases socket resources as if by POSIX `close(native_handle())`. Moves into `*this` the state originally represented by `rhs`.
 - 2 *Postconditions:*
 - (2.1) — `get_executor() == rhs.get_executor()`.
 - (2.2) — `is_open()` returns the same value as `rhs.is_open()` prior to the assignment.
 - (2.3) — `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the assignment.
 - (2.4) — `protocol_` is the prior value of `rhs.protocol_`.
 - (2.5) — `rhs.is_open() == false`.
 - 3 *Returns:* `*this`.
- ```
template<class OtherProtocol, class OtherExecutor>
basic_socket& operator=(basic_socket<OtherProtocol, OtherExecutor>&& rhs);
```
- 4 *Requires:* `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.
  - 5 *Effects:* If `is_open()` is `true`, cancels all outstanding asynchronous operations associated with this socket. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields `true`. Disables the linger socket option to prevent the assignment from blocking, and releases socket resources as if by POSIX `close(native_handle())`. Moves into `*this` the state originally represented by `rhs`.
  - 6 *Postconditions:*
    - (6.1) — `get_executor() == rhs.get_executor()`.
    - (6.2) — `is_open()` returns the same value as `rhs.is_open()` prior to the assignment.
    - (6.3) — `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the assignment.
    - (6.4) — `protocol_` is the result of converting the prior value of `rhs.protocol_`.
    - (6.5) — `rhs.is_open() == false`.
  - 7 *Returns:* `*this`.
  - 8 *Remarks:* This assignment operator shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

Update the class template `basic_datagram_socket` [`socket.basic`] as follows:

## 18.7 Class template `basic_datagram_socket`

[[socket.dgram](#)]

- 1 The class template `basic_datagram_socket<Protocol, Executor>` is used to send and receive discrete messages of fixed maximum length.



```

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

 template<class Protocol, class Executor>
 class basic_datagram_socket : public basic_socket<Protocol, Executor>
 {
 public:
 // types:

 using executor_type = Executor;
 using native_handle_type = implementation-defined; // see [socket.reqmts.native]
 using protocol_type = Protocol;
 using endpoint_type = typename protocol_type::endpoint;

 template<class OtherExecutor>
 using rebind_executor =
 basic_datagram_socket<Protocol, OtherExecutor>;

 // [socket.dgram.cons], construct / copy / destroy:

 explicit basic_datagram_socket(io_context& ctx& ex);
 template<class ExecutionContext>
 explicit basic_datagram_socket(ExecutionContext& ctx);
 basic_datagram_socket(io_context& ctx& ex, const protocol_type& protocol);
 template<class ExecutionContext>
 basic_datagram_socket(ExecutionContext& ctx, const protocol_type& protocol);
 basic_datagram_socket(io_context& ctx& ex, const endpoint_type& endpoint);
 template<class ExecutionContext>
 basic_datagram_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
 basic_datagram_socket(io_context& ctx& ex, const protocol_type& protocol,
 const native_handle_type& native_socket);
 template<class ExecutionContext>
 basic_datagram_socket(ExecutionContext& ctx, const protocol_type& protocol,
 const native_handle_type& native_socket);
 basic_datagram_socket(const basic_datagram_socket&) = delete;
 basic_datagram_socket(basic_datagram_socket&& rhs);
 template<class OtherProtocol, class OtherExecutor>
 basic_datagram_socket(basic_datagram_socket<OtherProtocol, OtherExecutor>&& rhs);

 ~basic_datagram_socket();

 basic_datagram_socket& operator=(const basic_datagram_socket&) = delete;
 basic_datagram_socket& operator=(basic_datagram_socket&& rhs);
 template<class OtherProtocol, class OtherExecutor>
 basic_datagram_socket& operator=(basic_datagram_socket<OtherProtocol, OtherExecutor>&& rhs);
 };
}
}
}
}

```

[...]

- <sup>6</sup> If native\_handle\_type and basic\_socket<Protocol, Executor>::native\_handle\_type are both defined then they name the same type.

Update the basic\_datagram\_socket constructors [socket.dgram.cons] as follows:

### 18.7.1 basic\_datagram\_socket constructors [socket.dgram.cons]

```
explicit basic_datagram_socket(io_context& ctx& ex);
```

- <sup>1</sup> *Effects:* Initializes the base class with basic\_socket<Protocol, Executor>(~~& ex~~).

```
template<class ExecutionContext>
explicit basic_datagram_socket(ExecutionContext& ctx);
```

*Effects:* Equivalent to basic\_datagram\_socket(ctx.get\_executor()).

*Remarks:* This function shall not participate in overload resolution unless is\_convertible<ExecutionContext&, execution\_context\_t::value> is true and is\_constructible<executor\_type, typename ExecutionContext::executor\_type>::value is true.

```
basic_datagram_socket(io_context& ctx& ex, const protocol_type& protocol);
```

*Effects:*Initializes the base class with `basic_socket<Protocol, Executor>(ctx, protocol)`.

```
template<class ExecutionContext>
explicit basic_datagram_socket(ExecutionContext& ctx, const protocol_type& protocol);
```

*Effects:*Equivalent to `basic_datagram_socket(ctx.get_executor(), protocol)`.

*Remarks:*This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is `true`.

```
basic_datagram_socket(io_context& ctxconst executor_type& ex, const endpoint_type& endpoint);
```

3 *Effects:*Initializes the base class with `basic_socket<Protocol, Executor>(ctx, endpoint)`.

```
template<class ExecutionContext>
explicit basic_datagram_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
```

*Effects:*Equivalent to `basic_datagram_socket(ctx.get_executor(), endpoint)`.

*Remarks:*This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is `true`.

```
basic_datagram_socket(io_context& ctx, const protocol_type& protocol,
 const native_handle_type& native_socket);
```

4 *Effects:*Initializes the base class with `basic_socket<Protocol, Executor>(ctx, protocol, native_socket)`.

```
template<class ExecutionContext>
basic_datagram_socket(ExecutionContext& ctx, const protocol_type& protocol,
 const native_handle_type& native_socket);
```

*Effects:*Equivalent to `basic_datagram_socket(ctx.get_executor(), protocol, native_socket)`.

*Remarks:*This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is `true`.

```
basic_datagram_socket(basic_datagram_socket&& rhs);
```

5 *Effects:*Move constructs an object of class `basic_datagram_socket<Protocol, Executor>`, initializing the base class with `basic_socket<Protocol, Executor>(std::move(rhs))`.

```
template<class OtherProtocol, class OtherExecutor>
basic_datagram_socket(basic_datagram_socket<OtherProtocol, OtherExecutor>&& rhs);
```

6 *Requires:*OtherProtocol is implicitly convertible to Protocol and OtherExecutor is implicitly convertible to Executor.

7 *Effects:*Move constructs an object of class `basic_datagram_socket<Protocol, Executor>`, initializing the base class with `basic_socket<Protocol, Executor>(std::move(rhs))`.

8 *Remarks:*This constructor shall not participate in overload resolution unless OtherProtocol is implicitly convertible to Protocol and OtherExecutor is implicitly convertible to Executor.

Update the `basic_datagram_socket` assignment operators [`socket.dgram.assign`] as follows:

## 18.7.2 basic\_datagram\_socket assignment

[[socket.dgram.assign](#)]

```
basic_datagram_socket& operator=(basic_datagram_socket&& rhs);
```

1 *Effects:*Equivalent to `basic_socket<Protocol, Executor>::operator=(std::move(rhs))`.

2 *Returns:*\*this.

```
template<class OtherProtocol, class OtherExecutor>
basic_datagram_socket& operator=(basic_datagram_socket<OtherProtocol, OtherExecutor>&& rhs);
```

3

Requires: `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

4 Effects: Equivalent to `basic_socket<Protocol, Executor>::operator=(std::move(rhs))`.

5 Returns: `*this`.

6 Remarks: This assignment operator not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

Update the class template `basic_stream_socket` [`socket.stream`] as follows:

## 18.8 Class template `basic_stream_socket` [`socket.stream`]

1 The class template `basic_stream_socket<Protocol, Executor>` is used to exchange data with a peer over a sequenced, reliable, bidirectional, connection-mode byte stream.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

template<class Protocol, class Executor>
class basic_stream_socket : public basic_socket<Protocol, Executor>
{
public:
 // types:

 using executor_type = Executor;
 using native_handle_type = implementation-defined; // see [socket.reqmts.native]
 using protocol_type = Protocol;
 using endpoint_type = typename protocol_type::endpoint;

 template<class OtherExecutor>
 using rebind_executor =
 basic_stream_socket<Protocol, OtherExecutor>;

 // [socket.stream.cons], construct / copy / destroy:

 explicit basic_stream_socket(io_context& ctx const executor_type& ex);
 template<class ExecutionContext>
 explicit basic_stream_socket(ExecutionContext& ctx);
 basic_stream_socket(io_context& ctx const executor_type& ex, const protocol_type& protocol);
 template<class ExecutionContext>
 basic_stream_socket(ExecutionContext& ctx, const protocol_type& protocol);
 basic_stream_socket(io_context& ctx const executor_type& ex, const endpoint_type& endpoint);
 template<class ExecutionContext>
 basic_stream_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
 basic_stream_socket(io_context& ctx const executor_type& ex, const protocol_type& protocol,
 const native_handle_type& native_socket);
 template<class ExecutionContext>
 basic_stream_socket(ExecutionContext& ctx, const protocol_type& protocol,
 const native_handle_type& native_socket);
 basic_stream_socket(const basic_stream_socket&) = delete;
 basic_stream_socket(basic_stream_socket&& rhs);
 template<class OtherProtocol, class OtherExecutor>
 basic_stream_socket(basic_stream_socket<OtherProtocol, OtherExecutor>&& rhs);

 ~basic_stream_socket();

 basic_stream_socket& operator=(const basic_stream_socket&) = delete;
 basic_stream_socket& operator=(basic_stream_socket&& rhs);
 template<class OtherProtocol, class OtherExecutor>
 basic_stream_socket& operator=(basic_stream_socket<OtherProtocol, OtherExecutor>&& rhs);

 [...]
```

6 If `native_handle_type` and `basic_socket<Protocol, Executor>::native_handle_type` are both defined then they name the same type.

Update the `basic_stream_socket` constructors [`socket.stream.cons`] as follows:

```
explicit basic_stream_socket(io_context& ctx const executor_type& ex);
```

- 1 *Effects:* Initializes the base class with `basic_socket<Protocol, Executor>(ctx)`.

```
template<class ExecutionContext>
explicit basic_stream_socket(ExecutionContext& ctx);
```

*Effects:* Equivalent to `basic_stream_socket(ctx.get_executor())`.

*Remarks:* This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution context&>::value is true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value is true`.

```
basic_stream_socket(io_context& ctx const executor_type& ex, const protocol_type& protocol);
```

- 2 *Effects:* Initializes the base class with `basic_socket<Protocol, Executor>(ctx, protocol)`.

```
template<class ExecutionContext>
explicit basic_stream_socket(ExecutionContext& ctx, const protocol_type& protocol);
```

*Effects:* Equivalent to `basic_stream_socket(ctx.get_executor(), protocol)`.

*Remarks:* This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution context&>::value is true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value is true`.

```
basic_stream_socket(io_context& ctx const executor_type& ex, const endpoint_type& endpoint);
```

- 3 *Effects:* Initializes the base class with `basic_socket<Protocol, Executor>(ctx, endpoint)`.

```
template<class ExecutionContext>
explicit basic_stream_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
```

*Effects:* Equivalent to `basic_stream_socket(ctx.get_executor(), endpoint)`.

*Remarks:* This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution context&>::value is true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value is true`.

```
basic_stream_socket(io_context& ctx const executor_type& ex, const protocol_type& protocol,
 const native_handle_type& native_socket);
```

- 4 *Effects:* Initializes the base class with `basic_socket<Protocol, Executor>(ctx, protocol, native_socket)`.

```
template<class ExecutionContext>
basic_stream_socket(ExecutionContext& ctx, const protocol_type& protocol,
 const native_handle_type& native_socket);
```

*Effects:* Equivalent to `basic_stream_socket(ctx.get_executor(), protocol, native_socket)`.

*Remarks:* This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution context&>::value is true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value is true`.

```
basic_stream_socket(basic_stream_socket&& rhs);
```

- 5 *Effects:* Move constructs an object of class `basic_stream_socket<Protocol, Executor>`, initializing the base class with `basic_socket<Protocol, Executor>(std::move(rhs))`.

```
template<class OtherProtocol, class OtherExecutor>
basic_stream_socket(basic_stream_socket<OtherProtocol, OtherExecutor>&& rhs);
```

- 6 *Requires:* `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

- 7 *Effects:* Move constructs an object of class `basic_stream_socket<Protocol, Executor>`, initializing the base class with `basic_socket<Protocol, Executor>(std::move(rhs))`.

Remarks: This constructor shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

Update the `basic_stream_socket` assignment operators [`socket.stream.assign`] as follows:

## 18.8.2 `basic_stream_socket` assignment [`socket.stream.assign`]

```
basic_stream_socket& operator=(basic_stream_socket&& rhs);
```

1 *Effects:* Equivalent to `basic_socket<Protocol, Executor>::operator=(std::move(rhs))`.

2 *Returns:* `*this`.

```
template<class OtherProtocol, class OtherExecutor>
basic_stream_socket& operator=(basic_stream_socket<OtherProtocol, OtherExecutor>&& rhs);
```

3 *Requires:* `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

4 *Effects:* Equivalent to `basic_socket<Protocol, Executor>::operator=(std::move(rhs))`.

5 *Returns:* `*this`.

6 *Remarks:* This assignment operator shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

Update the class template `basic_socket_acceptor` [`socket.acceptor`] as follows:

## 18.9 Class template `basic_socket_acceptor` [`socket.acceptor`]

1 An object of class template `basic_socket_acceptor<AcceptableProtocol, Executor>` is used to listen for, and queue, incoming socket connections. Socket objects that represent the incoming connections are dequeued by calling `accept` or `async_accept`.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

template<class AcceptableProtocol, class Executor>
class basic_socket_acceptor : public socket_base
{
public:
 // types:

 using executor_type = io_context::executor_typeExecutor;
 using native_handle_type = implementation-defined; // see [socket.reqmts.native]
 using protocol_type = AcceptableProtocol;
 using endpoint_type = typename protocol_type::endpoint;
 using socket_type = typename protocol_type::socket;
 template<class OtherExecutor>
 using socket_type_for
 = typename protocol_type::socket_for<OtherExecutor>;

 template<class OtherExecutor>
 using rebind_executor =
 basic_socket_acceptor<AcceptableProtocol, OtherExecutor>;

 // [socket.acceptor.cons], construct / copy / destroy:

 explicit basic_socket_acceptor(io_context& ex);
 template<class ExecutionContext>
 explicit basic_socket_acceptor(ExecutionContext& ctx);
 basic_socket_acceptor(io_context& ex, const protocol_type& protocol);
 template<class ExecutionContext>
 basic_socket_acceptor(ExecutionContext& ctx, const protocol_type& protocol);
 basic_socket_acceptor(io_context& ex, const endpoint_type& endpoint,
 bool reuse_addr = true);
 template<class ExecutionContext>
 basic_socket_acceptor(ExecutionContext& ctx, const endpoint_type& endpoint,
 bool reuse_addr = true);
```

```

basic_socket_acceptor(io_context& ctxconst_executor_type& ex, const protocol_type& protocol,
 const native_handle_type& native_socket);
template<class ExecutionContext>
basic_socket_acceptor(ExecutionContext& ctx, const protocol_type& protocol,
 const native_handle_type& native_socket);
basic_socket_acceptor(const basic_socket_acceptor&) = delete;
basic_socket_acceptor(basic_socket_acceptor&& rhs);
template<class OtherProtocol, class OtherExecutor>
basic_socket_acceptor(basic_socket_acceptor<OtherProtocol, OtherExecutor>&& rhs);

~basic_socket_acceptor();

basic_socket_acceptor& operator=(const basic_socket_acceptor&) = delete;
basic_socket_acceptor& operator=(basic_socket_acceptor&& rhs);
template<class OtherProtocol, class OtherExecutor>
basic_socket_acceptor& operator=(basic_socket_acceptor<OtherProtocol, OtherExecutor>&& rhs);

// [socket.acceptor.ops], basic_socket_acceptor operations:

executor_type get_executor() noexcept;

native_handle_type native_handle(); // see [socket.reqmts.native]

void open(const protocol_type& protocol = protocol_type());
void open(const protocol_type& protocol, error_code& ec);

void assign(const protocol_type& protocol,
 const native_handle_type& native_acceptor); // see [socket.reqmts.native]
void assign(const protocol_type& protocol,
 const native_handle_type& native_acceptor,
 error_code& ec); // see [socket.reqmts.native]

native_handle_type release(); // see [socket.reqmts.native]
native_handle_type release(error_code& ec); // see [socket.reqmts.native]

bool is_open() const noexcept;

void close();
void close(error_code& ec);

void cancel();
void cancel(error_code& ec);

template<class SettableSocketOption>
void set_option(const SettableSocketOption& option);
template<class SettableSocketOption>
void set_option(const SettableSocketOption& option, error_code& ec);

template<class GettableSocketOption>
void get_option(GettableSocketOption& option) const;
template<class GettableSocketOption>
void get_option(GettableSocketOption& option, error_code& ec) const;

template<class IoControlCommand>
void io_control(IoControlCommand& command);
template<class IoControlCommand>
void io_control(IoControlCommand& command, error_code& ec);

void non_blocking(bool mode);
void non_blocking(bool mode, error_code& ec);
bool non_blocking() const;

void native_non_blocking(bool mode);
void native_non_blocking(bool mode, error_code& ec);
bool native_non_blocking() const;

void bind(const endpoint_type& endpoint);
void bind(const endpoint_type& endpoint, error_code& ec);

void listen(int backlog = max_listen_connections);
void listen(int backlog, error_code& ec);

endpoint_type local_endpoint() const;
endpoint_type local_endpoint(error_code& ec) const;

```

```

void enable_connection_aborted(bool mode);
bool enable_connection_aborted() const;

socket_type accept();
socket_type accept(error_code& ec);
socket_type accept(io_context& ctx);
socket_type accept(io_context& ctx, error_code& ec);
template<class OtherExecutor>
 socket_type for<OtherExecutor>
 accept(const OtherExecutor& ex);
template<class OtherExecutor>
 socket_type for<OtherExecutor>
 accept(const OtherExecutor& ex, error_code& ec);
template<class ExecutionContext>
 socket_type for<typename ExecutionContext::executor_type>
 accept(ExecutionContext& ctx);
template<class ExecutionContext>
 socket_type for<typename ExecutionContext::executor_type>
 accept(ExecutionContext& ctx, error_code& ec);

template<class CompletionToken>
 DEDUCED async_accept(CompletionToken&& token);
template<class CompletionToken>
DEDUCED async_accept(io_context& ctx, CompletionToken&& token);
template<class OtherExecutor, class CompletionToken>
 DEDUCED async_accept(const OtherExecutor& ex, CompletionToken&& token);
template<class ExecutionContext, class CompletionToken>
 DEDUCED async_accept(ExecutionContext& ctx, CompletionToken&& token);

socket_type accept(endpoint_type& endpoint);
socket_type accept(endpoint_type& endpoint, error_code& ec);
socket_type accept(io_context& ctx, endpoint_type& endpoint);
socket_type accept(io_context& ctx, endpoint_type& endpoint,
error_code& ec);
template<class OtherExecutor>
 socket_type for<OtherExecutor>
 accept(const OtherExecutor& ex, endpoint_type& endpoint);
template<class OtherExecutor>
 socket_type for<OtherExecutor>
 accept(const OtherExecutor& ex, endpoint_type& endpoint, error_code& ec);
template<class ExecutionContext>
 socket_type for<typename ExecutionContext::executor_type>
 accept(ExecutionContext& ctx, endpoint_type& endpoint);
template<class ExecutionContext>
 socket_type for<typename ExecutionContext::executor_type>
 accept(ExecutionContext& ctx, endpoint_type& endpoint, error_code& ec);

template<class CompletionToken>
 DEDUCED async_accept(endpoint_type& endpoint,
 CompletionToken&& token);
template<class CompletionToken>
DEDUCED async_accept(io_context& ctx, endpoint_type& endpoint,
CompletionToken&& token);
template<class OtherExecutor, class CompletionToken>
 DEDUCED async_accept(const OtherExecutor& ex, endpoint_type& endpoint,
 CompletionToken&& token);
template<class ExecutionContext, class CompletionToken>
 DEDUCED async_accept(ExecutionContext& ctx, endpoint_type& endpoint,
 CompletionToken&& token);

```

Update the `basic_socket_acceptor` constructors [`socket.stream.cons`] as follows:

### 18.9.1 basic\_socket\_acceptor constructors

[[socket.acceptor.cons](#)]

```
explicit basic_socket_acceptor(io_context& ctxconst executor_type& ex);
```

1 Postconditions:

(1.1) — `get_executor()` == ~~`ctx.get_executor()`~~`ex`.

(1.2) — `is_open()` == `false`.



```
template<class ExecutionContext>
explicit basic_socket_acceptor(ExecutionContext& ctx);
Effects:Equivalent to basic_socket_acceptor(ctx.get_executor().).
Remarks:This function shall not participate in overload resolution unless is -
convertible<ExecutionContext&, execution context&>::value is true and is -
constructible<executor_type, typename ExecutionContext::executor_type>::value is true.
```

```
basic_socket_acceptor(io_context& ctxconst executor_type& ex, const protocol_type& protocol);
```

2 *Effects:*Opens this acceptor as if by calling `open(protocol)`.

3 *Postconditions:*

(3.1) — `get_executor() == ctx.get_executor()ex`.

(3.2) — `is_open() == true`.

(3.3) — `non_blocking() == false`.

(3.4) — `enable_connection_aborted() == false`.

(3.5) — `protocol_ == protocol`.

```
template<class ExecutionContext>
explicit basic_socket_acceptor(ExecutionContext& ctx, const protocol_type& protocol);
```

*Effects:*Equivalent to `basic_socket_acceptor(ctx.get_executor(),_protocol)`.

*Remarks:*This function shall not participate in overload resolution unless `is -
convertible<ExecutionContext&, execution context&>::value is true` and `is -
constructible<executor_type, typename ExecutionContext::executor_type>::value is true`.

```
basic_socket_acceptor(io_context& ctxconst executor_type& ex, const endpoint_type& endpoint,
bool reuse_addr = true);
```

4 *Effects:*Opens and binds this acceptor as if by calling:

```
open(endpoint.protocol());
if (reuse_addr)
 set_option(reuse_address(true));
bind(endpoint);
listen();
```

5 *Postconditions:*

(5.1) — `get_executor() == ctx.get_executor()ex`.

(5.2) — `is_open() == true`.

(5.3) — `non_blocking() == false`.

(5.4) — `enable_connection_aborted() == false`.

(5.5) — `protocol_ == endpoint.protocol()`.

```
template<class ExecutionContext>
explicit basic_socket_acceptor(ExecutionContext& ctx, const endpoint_type& endpoint,
bool reuse_addr = true);
```

*Effects:*Equivalent to `basic_socket_acceptor(ctx.get_executor(),_endpoint, reuse_addr)`.

*Remarks:*This function shall not participate in overload resolution unless `is -
convertible<ExecutionContext&, execution context&>::value is true` and `is -
constructible<executor_type, typename ExecutionContext::executor_type>::value is true`.

```
basic_socket_acceptor(io_context& ctxconst executor_type& ex, const protocol_type& protocol,
const native_handle_type& native_acceptor);
```

6 *Requires:*`native_acceptor` is a native handle to an open acceptor.

7 *Effects:*Assigns the existing native acceptor into this acceptor as if by calling `assign(protocol, native_
acceptor)`.

8 *Postconditions:*



- (8.1) — `get_executor()` == `ctx.get_executor().ex`.
- (8.2) — `is_open()` == `true`.
- (8.3) — `non_blocking()` == `false`.
- (8.4) — `enable_connection_aborted()` == `false`.
- (8.5) — `protocol_` == `protocol`.

```
template<class ExecutionContext>
basic_socket_acceptor(ExecutionContext& ctx, const protocol_type& protocol,
 const native_handle_type& native_socket);
```

Effects: Equivalent to `basic_socket_acceptor(ctx.get_executor(), _protocol, native_socket)`.  
Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>` is `true` and `is_constructible<executor_type, typename ExecutionContext::executor_type>` is `true`.

```
basic_socket_acceptor(basic_socket_acceptor&& rhs);
```

9 *Effects:* Move constructs an object of class `basic_socket_acceptor<AcceptableProtocol, Executor>` that refers to the state originally represented by `rhs`.

10 *Postconditions:*

- (10.1) — `get_executor()` == `rhs.get_executor()`.
- (10.2) — `is_open()` returns the same value as `rhs.is_open()` prior to the constructor invocation.
- (10.3) — `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the constructor invocation.
- (10.4) — `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the constructor invocation.
- (10.5) — `native_handle()` returns the same value as `rhs.native_handle()` prior to the constructor invocation.
- (10.6) — `protocol_` is equal to the prior value of `rhs.protocol_`.
- (10.7) — `rhs.is_open()` == `false`.

```
template<class OtherProtocol, class OtherExecutor>
basic_socket_acceptor(basic_socket_acceptor<OtherProtocol, OtherExecutor>&& rhs);
```

11 *Requires:* `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

12 *Effects:* Move constructs an object of class `basic_socket_acceptor<AcceptableProtocol, Executor>` that refers to the state originally represented by `rhs`.

13 *Postconditions:*

- (13.1) — `get_executor()` == `rhs.get_executor()`.
- (13.2) — `is_open()` returns the same value as `rhs.is_open()` prior to the constructor invocation.
- (13.3) — `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the constructor invocation.
- (13.4) — `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the constructor invocation.
- (13.5) — `native_handle()` returns the prior value of `rhs.native_handle()`.
- (13.6) — `protocol_` is the result of converting the prior value of `rhs.protocol_`.
- (13.7) — `rhs.is_open()` == `false`.

14 *Remarks:* This constructor shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

Update the `basic_socket_acceptor` assignment operators [`socket.stream.assign`] as follows:

### 18.9.3

### `basic_socket_acceptor` assignment

[[socket.acceptor.assign](#)]

```
basic_socket_acceptor& operator=(basic_socket_acceptor&& rhs);
```

1 *Effects:*If `is_open()` is `true`, cancels all outstanding asynchronous operations associated with this acceptor, and releases acceptor resources as if by POSIX `close(native_handle())`. Then moves into `*this` the state originally represented by `rhs`. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields `true`.

2 *Postconditions:*

- (2.1) — `get_executor()` == `rhs.get_executor()`.
- (2.2) — `is_open()` returns the same value as `rhs.is_open()` prior to the assignment.
- (2.3) — `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the assignment.
- (2.4) — `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the assignment.
- (2.5) — `native_handle()` returns the same value as `rhs.native_handle()` prior to the assignment.
- (2.6) — `protocol_` is the same value as `rhs.protocol_` prior to the assignment.
- (2.7) — `rhs.is_open() == false`.

3 *Returns:*`*this`.

```
template<class OtherProtocol, class OtherExecutor>
basic_socket_acceptor& operator=(basic_socket_acceptor<OtherProtocol, OtherExecutor>&& rhs);
```

4 *Requires:*OtherProtocol is implicitly convertible to Protocol and OtherExecutor is implicitly convertible to Executor.

5 *Effects:*If `is_open()` is `true`, cancels all outstanding asynchronous operations associated with this acceptor, and releases acceptor resources as if by POSIX `close(native_handle())`. Then moves into `*this` the state originally represented by `rhs`. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields `true`.

6 *Postconditions:*

- (6.1) — `get_executor()` == `rhs.get_executor()`.
- (6.2) — `is_open()` returns the same value as `rhs.is_open()` prior to the assignment.
- (6.3) — `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the assignment.
- (6.4) — `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the assignment.
- (6.5) — `native_handle()` returns the same value as `rhs.native_handle()` prior to the assignment.
- (6.6) — `protocol_` is the result of converting the value of `rhs.protocol_` prior to the assignment.
- (6.7) — `rhs.is_open() == false`.

7 *Returns:*`*this`.

8 *Remarks:*This assignment operator shall not participate in overload resolution unless OtherProtocol is implicitly convertible to Protocol and OtherExecutor is implicitly convertible to Executor.

Update the `basic_socket_acceptor` operations [socket.stream.ops] as follows:

## 18.9.4 basic\_socket\_acceptor operations

[socket.acceptor.ops]

[...]

```
socket_type accept();
socket_type accept(error_code& ec);
```

40 *Returns:*`accept(get_executor().context(), ec)`.

```
socket_type accept(io_context& ctx);
socket_type accept(io_context& ctx, error_code& ec);
template<class OtherExecutor>
```

```

 socket_type_for<OtherExecutor>
 accept(const OtherExecutor& ex);
template<class OtherExecutor>
 socket_type_for<OtherExecutor>
 accept(const OtherExecutor& ex, error_code& ec);

```

41 *Effects:*Extracts a socket from the queue of pending connections of the acceptor, as if by POSIX:

```
native_handle_type h = accept(native_handle(), nullptr, 0);
```

42 *Returns:*On success, `socket_type_for<OtherExecutor>(et*ex, protocol_, h)`. Otherwise `socket_type_for<OtherExecutor>(et*ex)`.

```

template<class ExecutionContext>
 socket_type_for<typename ExecutionContext::executor_type>
 accept(ExecutionContext& ctx);
template<class ExecutionContext>
 socket_type_for<typename ExecutionContext::executor_type>
 accept(ExecutionContext& ctx, error_code& ec);

```

*Returns:*`accept(ctx.get_executor()., _ec)`.

```

template<class CompletionToken>
 DEDUCED async_accept(CompletionToken&& token);

```

43 *Returns:*

```
async_accept(get_executor().context(), forward<CompletionToken>(token))
```

```

template<class CompletionToken>
DEDUCED async_accept(io_context& ctx, CompletionToken&& token);
template<class OtherExecutor, class CompletionToken>
DEDUCED async_accept(const OtherExecutor& ex, CompletionToken&& token);

```

Let S be the type `socket_type_for<OtherExecutor>`.

44 *Completion signature:*`void(error_code ec, socket_type S s)`.

45 *Effects:*Initiates an asynchronous operation to extract a socket from the queue of pending connections of the acceptor, as if by POSIX:

```
native_handle_type h = accept(native_handle(), nullptr, 0);
```

~~On success, s is `socket_type(ctx, protocol_, h)`. Otherwise, s is `socket_type(ctx)`.~~ On success, s is `S(ex, protocol_, h)`. Otherwise, s is `S(ex)`.

```

template<class ExecutionContext, class CompletionToken>
 DEDUCED async_accept(ExecutionContext& ctx, CompletionToken&& token);

```

*Returns:*

```
async_accept(ctx.get_executor()., forward<CompletionToken>(token)).
```

```

socket_type accept(endpoint_type& endpoint);
socket_type accept(endpoint_type& endpoint, error_code& ec);

```

46 *Returns:*`accept(get_executor().context(), endpoint, ec)`.

```

socket_type accept(io_context& ctx, endpoint_type& endpoint);
socket_type accept(io_context& ctx, endpoint_type& endpoint,
error_code& ec);

```

```

template<class OtherExecutor>
 socket_type_for<OtherExecutor>
 accept(const OtherExecutor& ex, endpoint_type& endpoint);
template<class OtherExecutor>
 socket_type_for<OtherExecutor>
 accept(const OtherExecutor& ex, endpoint_type& endpoint, error_code& ec);

```

47 *Effects:*Extracts a socket from the queue of pending connections of the acceptor, as if by POSIX:

```

socklen_t endpoint_len = endpoint.capacity();
native_handle_type h = accept(native_handle(),
 endpoint.data(),

```

```

 &endpoint_len);
 if (h >= 0)
 endpoint.resize(endpoint_len);

```

48 *Returns:* On success, `socket_type(ctxex, protocol_, h)`. Otherwise `socket_type(ctxex)`.

```

template<class ExecutionContext>
 socket_type for<typename ExecutionContext::executor_type>
 accept(ExecutionContext& ctx, endpoint_type& endpoint);
template<class ExecutionContext>
 socket_type for<typename ExecutionContext::executor_type>
 accept(ExecutionContext& ctx, endpoint_type& endpoint, error_code& ec);

```

*Returns:* `accept(ctx.get_executor(), endpoint, ec)`.

```

template<class CompletionToken>
 DEDUCED async_accept(endpoint_type& endpoint,
 CompletionToken&& token);

```

49 *Returns:*

```

 async_accept(get_executor().context(), endpoint, forward<CompletionToken>(token))

```

```

template<class CompletionToken>
 DEDUCED async_accept(io_context& ctx, endpoint_type& endpoint,
 CompletionToken&& token);
template<class OtherExecutor, class CompletionToken>
 DEDUCED async_accept(const OtherExecutor& ex, endpoint_type& endpoint,
 CompletionToken&& token);

```

*Let S be the type socket\_type for<OtherExecutor>.*

50 *Completion signature:* `void(error_code ec, socket_type S s)`.

51 *Effects:* Initiates an asynchronous operation to extract a socket from the queue of pending connections of the acceptor, as if by POSIX:

```

 socklen_t endpoint_len = endpoint.capacity();
 native_handle_type h = accept(native_handle(),
 endpoint.data(),
 &endpoint_len);

 if (h >= 0)
 endpoint.resize(endpoint_len);

```

~~On success, s is socket\_type(ctx, protocol\_, h). Otherwise, s is socket\_type(ctx).~~ *On success, s is S(ex, protocol\_, h). Otherwise, s is S(ex).*

```

template<class ExecutionContext, class CompletionToken>
 DEDUCED async_accept(ExecutionContext& ctx, endpoint_type& endpoint,
 CompletionToken&& token);

```

*Returns:*

```

 async_accept(ctx.get_executor(), forward<CompletionToken>(token))

```

Update the class template `basic_socket_streambuf` [`socket.streambuf`] as follows:

## 19.1 Class template `basic_socket_streambuf` [`socket.streambuf`]

- 1 The class `basic_socket_streambuf<Protocol, Clock, WaitTraits, Executor>` associates both the input sequence and the output sequence with a socket. The input and output sequences do not support seeking. [ *Note:* The input and output sequences are independent as a stream socket provides full duplex I/O. — *end note* ]
- 2 [ *Note:* This class is intended for sending and receiving bytes, not characters. Any conversion from characters to bytes, and vice versa, occurs elsewhere. — *end note* ]

```

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

```

```

template<class Protocol, class Clock, class WaitTraits, class Executor = executor>
class basic_socket_streambuf : public basic_streambuf<char>
{
public:
 // types:

 using executor_type = Executor;
 using protocol_type = Protocol;
 using endpoint_type = typename protocol_type::endpoint;
 using clock_type = Clock;
 using time_point = typename clock_type::time_point;
 using duration = typename clock_type::duration;
 using wait_traits_type = WaitTraits;

 // [socket.streambuf.cons], construct / copy / destroy:

 basic_socket_streambuf();
 explicit basic_socket_streambuf(basic_stream_socket<protocol_type, executor_type> s);
 basic_socket_streambuf(const basic_socket_streambuf&) = delete;
 basic_socket_streambuf(basic_socket_streambuf&& rhs);

 virtual ~basic_socket_streambuf();

 basic_socket_streambuf& operator=(const basic_socket_streambuf&) = delete;
 basic_socket_streambuf& operator=(basic_socket_streambuf&& rhs);

 // [socket.streambuf.members], members:

 basic_socket_streambuf* connect(const endpoint_type& e);
 template<class... Args> basic_socket_streambuf* connect(Args&&...);

 basic_socket_streambuf* close();

 basic_socket<protocol_type, executor_type>& socket();
 error_code error() const;

 time_point expiry() const;
 void expires_at(const time_point& t);
 void expires_after(const duration& d);

protected:
 // overridden virtual functions:
 virtual int_type underflow() override;
 virtual int_type pbackfail(int_type c = traits_type::eof()) override;
 virtual int_type overflow(int_type c = traits_type::eof()) override;
 virtual int sync() override;
 virtual streambuf* setbuf(char_type* s, streamsize n) override;

private:
 basic_stream_socket<protocol_type, executor_type> socket_; // exposition only
 error_code ec_; // exposition only
 time_point expiry_; // exposition only
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

- 3 Instances of class template `basic_socket_streambuf` meet the requirements of `Destructible` (C++2014[destructible]), `MoveConstructible` (C++2014[moveconstructible]), and `MoveAssignable` (C++2014[moveassignable]).

### 19.1.1 `basic_socket_streambuf` constructors

[\[socket.streambuf.cons\]](#)

`basic_socket_streambuf();`

- 1 *Effects:* Initializes `socket_` with `ctx`, where `ctx` is an unspecified object of class `io_context`.
- 2 *Postconditions:* `expiry() == time_point::max()` and `!error()`.

Remarks:This function shall not participate in overload resolution unless `is_constructible<executor - type, io_context::executor_type>::value is true`.

```
explicit basic_socket_streambuf(basic_stream_socket<protocol_type, executor_type> s);
```

3     *Effects:*Initializes `socket_` with `std::move(s)`.

4     *Postconditions:*`expiry() == time_point::max()` and `!error()`.

```
basic_socket_streambuf(basic_socket_streambuf&& rhs);
```

5     *Effects:*Move constructs from the rvalue `rhs`. It is implementation-defined whether the sequence pointers in `*this` (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) obtain the values which `rhs` had. Whether they do or not, `*this` and `rhs` reference separate buffers (if any at all) after the construction. Additionally `*this` references the socket which `rhs` did before the construction, and `rhs` references no open socket after the construction.

6     *Postconditions:*Let `rhs_p` refer to the state of `rhs` just prior to this construction and let `rhs_a` refer to the state of `rhs` just after this construction.

(6.1)     `__ is_open() == rhs_p.is_open()`

(6.2)     `__ rhs_a.is_open() == false`

(6.3)     `__ error() == rhs_p.error()`

(6.4)     `__ !rhs_a.error()`

(6.5)     `__ expiry() == rhs_p.expiry()`

(6.6)     `__ rhs_a.expiry() == time_point::max()`

(6.7)     `__ gptr() - eback() == rhs_p.gptr() - rhs_p.eback()`

(6.8)     `__ egptr() - eback() == rhs_p.egptr() - rhs_p.eback()`

(6.9)     `__ ptr() - pbase() == rhs_p.pptr() - rhs_p.pbase()`

(6.10)    `__ pptr() - pbase() == rhs_p.epptr() - rhs_p.pbase()`

(6.11)    `__ if (eback()) eback() != rhs_a.eback()`

(6.12)    `__ if (gptr()) gptr() != rhs_a.gptr()`

(6.13)    `__ if (egptr()) egptr() != rhs_a.egptr()`

(6.14)    `__ if (pbase()) pbase() != rhs_a.pbase()`

(6.15)    `__ if (pptr()) pptr() != rhs_a.pptr()`

(6.16)    `__ if (epptr()) epptr() != rhs_a.epptr()`

```
virtual ~basic_socket_streambuf();
```

7     *Effects:*If a put area exists, calls `overflow(traits_type::eof())` to flush characters. [ *Note:* The socket is closed by the `basic_stream_socket<protocol_type, executor_type>` destructor. — *end note* ]

```
basic_socket_streambuf& operator=(basic_socket_streambuf&& rhs);
```

8     *Effects:*Calls `this->close()` then move assigns from `rhs`. After the move assignment `*this` and `rhs` have the observable state they would have had if `*this` had been move constructed from `rhs`.

9     *Returns:*`*this`.

## 19.1.2           basic\_socket\_streambuf members

[[socket.streambuf.members](#)]

```
basic_socket_streambuf* connect(const endpoint_type& e);
```

1     *Effects:*Initializes the `basic_socket_streambuf` as required, closes and re-opens the socket by performing `socket_.close(ec_)` and `socket_.open(e.protocol(), ec_)`, then attempts to establish a connection as if by POSIX `connect(socket_.native_handle(), static_cast<sockaddr*>(e.data()), e.size())`. `ec_` is set to reflect the error code produced by the operation. If the operation does not complete before the absolute timeout specified by `expiry_`, the socket is closed and `ec_` is set to `errc::timed_out`.

2     *Returns:*if `!ec_`, `this`; otherwise, a null pointer.

```
template<class... Args>
basic_socket_streambuf* connect(Args&&... args);
```

- 3 *Effects:* Initializes the `basic_socket_streambuf` as required and closes the socket as if by calling `socket_.close(ec_)`. Obtains an endpoint sequence endpoints by performing `protocol_type::resolver(ctx).resolve(forward<Args>(args)...)...`, where `ctx` is an unspecified object of class `io_context`. For each endpoint `e` in the sequence, closes and re-opens the socket by performing `socket_.close(ec_)` and `socket_.open(e.protocol(), ec_)`, then attempts to establish a connection as if by POSIX `connect(socket_.native_handle(), static_cast<sockaddr*>(e.data()), e.size())`. `ec_` is set to reflect the error code produced by the operation. If the operation does not complete before the absolute timeout specified by `expiry_`, the socket is closed and `ec_` is set to `errc::timed_out`.
- 4 *Returns:* if `!ec_`, `this`; otherwise, a null pointer.
- 5 *Remarks:* This function shall not participate in overload resolution unless `Protocol` meets the requirements for an internet protocol ([\[internet.reqmts.protocol\]](#)).

```
basic_socket_streambuf* close();
```

- 6 *Effects:* If a put area exists, calls `overflow(traits_type::eof())` to flush characters. Regardless of whether the preceding call fails or throws an exception, the function closes the socket as if by `basic_socket<protocol_type, _executor_type>::close(ec_)`. If any of the calls made by the function fail, `close` fails by returning a null pointer. If one of these calls throws an exception, the exception is caught and rethrown after closing the socket.
- 7 *Returns:* `this` on success, a null pointer otherwise.
- 8 *Postconditions:* `is_open() == false`.

```
basic_socket<protocol_type, _executor_type>& socket();
```

- 9 *Returns:* `socket_`.

Update the class template `basic_socket_iostream` [`socket.iostream`] as follows:

## 19.2 Class template `basic_socket_iostream` [\[socket.iostream\]](#)

- 1 The class template `basic_socket_iostream<Protocol, Clock, WaitTraits, Executor>` supports reading and writing on sockets. It uses a `basic_socket_streambuf<Protocol, Clock, WaitTraits, Executor>` object to control the associated sequences.
- 2 [ *Note:* This class is intended for sending and receiving bytes, not characters. Any conversion from characters to bytes, and vice versa, occurs elsewhere. — *end note* ]

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

template<class Protocol, class Clock, class WaitTraits, class Executor = executor>
class basic_socket_iostream : public basic_iostream<char>
{
public:
 // types:

 using executor_type = Executor;
 using protocol_type = Protocol;
 using endpoint_type = typename protocol_type::endpoint;
 using clock_type = Clock;
 using time_point = typename clock_type::time_point;
 using duration = typename clock_type::duration;
 using wait_traits_type = WaitTraits;

 // [socket.iostream.cons], construct / copy / destroy:

 basic_socket_iostream();
 explicit basic_socket_iostream(basic_stream_socket<protocol_type, _executor_type> s);
 basic_socket_iostream(const basic_socket_iostream&) = delete;
 basic_socket_iostream(basic_socket_iostream&& rhs);
```

```

template<class... Args>
 explicit basic_socket_iostream(Args&&... args);

basic_socket_iostream& operator=(const basic_socket_iostream&) = delete;
basic_socket_iostream& operator=(basic_socket_iostream&& rhs);

// [socket.iostream.members], members:

template<class... Args> void connect(Args&&... args);

void close();

basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>* rdbuf() const;

basic_socket<protocol_type, executor_type>& socket();
error_code error() const;

time_point expiry() const;
void expires_at(const time_point& t);
void expires_after(const duration& d);

private:
 basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type> sb_; // exposition only
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

- 3 Instances of class template `basic_socket_iostream` meet the requirements of `Destructible` (C++2014[destructible]), `MoveConstructible` (C++2014[moveconstructible]), and `MoveAssignable` (C++2014[moveassignable]).

### 19.2.1 `basic_socket_iostream` constructors

[\[socket.iostream.cons\]](#)

```
basic_socket_iostream();
```

- 1 *Effects:* Initializes the base class as `basic_iostream<char>(&sb_)`, value-initializes `sb_`, and performs `setf(std::ios_base::unitbuf)`.

Remarks: This function shall not participate in overload resolution unless `is_default_constructible<basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>>::value` is `true`.

```
explicit basic_socket_iostream(basic_stream_socket<protocol_type, executor_type> s);
```

- 2 *Effects:* Initializes the base class as `basic_iostream<char>(&sb_)`, initializes `sb_` with `std::move(s)`, and performs `setf(std::ios_base::unitbuf)`.

```
basic_socket_iostream(basic_socket_iostream&& rhs);
```

- 3 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_socket_streambuf`. Next `basic_iostream<char>::set_rdbuf(&sb_)` is called to install the contained `basic_socket_streambuf`.

```
template<class... Args>
 explicit basic_socket_iostream(Args&&... args);
```

- 4 *Effects:* Initializes the base class as `basic_iostream<char>(&sb_)`, value-initializes `sb_`, and performs `setf(std::ios_base::unitbuf)`. Then calls `rdbuf()->connect(forward<Args>(args)...)...`. If that function returns a null pointer, calls `setstate(failbit)`.

```
basic_socket_iostream& operator=(basic_socket_iostream&& rhs);
```

- 5 *Effects:* Move assigns the base and members of `*this` from the base and corresponding members of `rhs`.

- 6 *Returns:* `*this`.

### 19.2.2 `basic_socket_iostream` members

[\[socket.iostream.members\]](#)



```

template<class... Args>
 void connect(Args&&... args);

1 Effects:Calls rdbuf()->connect(forward<Args>(args)...) . If that function returns a null pointer, calls
 setstate(failbit) (which may throw ios_base::failure).

void close();

2 Effects:Calls rdbuf()->close() . If that function returns a null pointer, calls setstate(failbit) (which may
 throw ios_base::failure).

basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>* rdbuf() const;

3 Let SB be the type basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type> .

4 Returns:const_cast<SB*>(addressof(sb_)) .

basic_socket<protocol_type, executor_type& socket();

5 Returns:rdbuf()->socket() .

```

Update the `connect` function [`socket.algo.connect`] as follows:

## 20.1 Synchronous connect operations [`socket.algo.connect`]

```

template<class Protocol, class Executor, class EndpointSequence>
 typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
 const EndpointSequence& endpoints);
template<class Protocol, class Executor, class EndpointSequence>
 typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
 const EndpointSequence& endpoints,
 error_code& ec);

1 Returns:connect(s, endpoints, [](auto, auto){ return true; }, ec) .

template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition>
 typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
 const EndpointSequence& endpoints,
 ConnectCondition c);
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition>
 typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
 const EndpointSequence& endpoints,
 ConnectCondition c, error_code& ec);

2 Effects:Performs ec.clear() , then finds the first element ep in the sequence endpoints for which:
(2.1) — c(ec, ep) yields true;
(2.2) — s.close(ec) succeeds;
(2.3) — s.open(ep.protocol(), ec) succeeds; and
(2.4) — s.connect(ep, ec) succeeds.

3 Returns:typename Protocol::endpoint() if no such element is found, otherwise ep .

4 Error conditions:
(4.1) — socket_errc::not_found — if endpoints.empty() or if the function object c returned false for all
 elements in the sequence.

template<class Protocol, class Executor, class InputIterator>
 InputIterator connect(basic_socket<Protocol, Executor>& s,
 InputIterator first, InputIterator last);
template<class Protocol, class Executor, class InputIterator>
 InputIterator connect(basic_socket<Protocol, Executor>& s,
 InputIterator first, InputIterator last,
 error_code& ec);

5 Returns:connect(s, first, last, [](auto, auto){ return true; }, ec) .

```

```

template<class Protocol, class Executor, class InputIterator, class ConnectCondition>
 InputIterator connect(basic_socket<Protocol, Executor>& s,
 InputIterator first, InputIterator last,
 ConnectCondition c);
template<class Protocol, class Executor, class InputIterator, class ConnectCondition>
 InputIterator connect(basic_socket<Protocol, Executor>& s,
 InputIterator first, InputIterator last,
 ConnectCondition c, error_code& ec);

```

6 *Effects:* Performs `ec.clear()`, then finds the first iterator `i` in the range `[first, last)` for which:

(6.1) — `c(ec, *i)` yields `true`;

(6.2) — `s.close(ec)` succeeds;

(6.3) — `s.open(typename Protocol::endpoint(*i).protocol(), ec)` succeeds; and

(6.4) — `s.connect(*i, ec)` succeeds.

7 *Returns:* last if no such iterator is found, otherwise `i`.

8 *Error conditions:*

(8.1) — `socket_errc::not_found` — if `first == last` or if the function object `c` returned `false` for all iterators in the range.

Update the `async_connect` function [`socket.algo.async.connect`] as follows:

## 20.2 Asynchronous connect operations [`socket.algo.async.connect`]

```

template<class Protocol, class Executor, class EndpointSequence, class CompletionToken>
 DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
 const EndpointSequence& endpoints,
 CompletionToken&& token);

```

1 *Returns:*

`async_connect(s, endpoints, [](auto, auto){ return true; }, forward<CompletionToken>(token))`

```

template<class Protocol, class Executor, class EndpointSequence,
 class ConnectCondition, class CompletionToken>
 DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
 const EndpointSequence& endpoints,
 ConnectCondition c,
 CompletionToken&& token);

```

2 A composed asynchronous operation ([`async.reqmts.async.composed`]).

3 *Completion signature:* `void(error_code ec, typename Protocol::endpoint ep)`.

4 *Effects:* Performs `ec.clear()`, then finds the first element `ep` in the sequence `endpoints` for which:

(4.1) — `c(ec, ep)` yields `true`;

(4.2) — `s.close(ec)` succeeds;

(4.3) — `s.open(ep.protocol(), ec)` succeeds; and

(4.4) — the asynchronous operation `s.async_connect(ep, unspecified)` succeeds.

5 `ec` is updated with the result of the `s.async_connect(ep, unspecified)` operation, if any. If no such element is found, or if the operation fails with one of the error conditions listed below, `ep` is set to `typename Protocol::endpoint()`. [ *Note:* The underlying `close`, `open`, and `async_connect` operations are performed sequentially. — *end note* ]

6 *Error conditions:*

(6.1) — `socket_errc::not_found` — if `endpoints.empty()` or if the function object `c` returned `false` for all elements in the sequence.

(6.2) — `errc::operation_canceled` — if `s.is_open() == false` immediately following an `async_connect` operation on the underlying socket.

```
template<class Protocol, class Executor, class InputIterator, class CompletionToken>
 DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
 InputIterator first, InputIterator last,
 CompletionToken&& token);
```

7 Returns:

```
 async_connect(s, first, last, [](auto, auto){ return true; },
 forward<CompletionToken>(token))
```

```
template<class Protocol, class Executor, class InputIterator,
 class ConnectCondition, class CompletionToken>
 DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
 InputIterator first, InputIterator last,
 ConnectCondition c,
 CompletionToken&& token);
```

8 A composed asynchronous operation ([[async.reqmts.async.composed](#)]).

9 Completion signature: `void(error_code ec, InputIterator i)`.

10 Effects: Performs `ec.clear()`, then finds the first iterator `i` in the range `[first, last)` for which:

(10.1) — `c(ec, *i)` yields `true`;

(10.2) — `s.close(ec)` succeeds;

(10.3) — `s.open(typename Protocol::endpoint(*i).protocol(), ec)` succeeds; and

(10.4) — the asynchronous operation `s.async_connect(*i, unspecified)` succeeds.

11 `ec` is updated with the result of the `s.async_connect(*i, unspecified)` operation, if any. If no such iterator is found, or if the operation fails with one of the error conditions listed below, `i` is set to `last`. [ [Note](#): The underlying `close`, `open`, and `async_connect` operations are performed sequentially. — *end note* ]

12 Error conditions:

(12.1) — `socket_errc::not_found` — if `first == last` or if the function object `c` returned `false` for all iterators in the range.

(12.2) — `errc::operation_canceled` — if `s.is_open() == false` immediately following an `async_connect` operation on the underlying socket.

Update the `<experimental/internet>` synopsis [[internet.synop](#)] as follows:

## 21.1 Header `<experimental/internet>` synopsis [[internet.synop](#)]

[...]

```
template<class InternetProtocol, class Executor = executor>
 class basic_resolver;
```

Update the class template `basic_resolver` [[internet.resolver](#)] as follows:

## 21.17 Class template `ip::basic_resolver` [[internet.resolver](#)]

1 Objects of type `basic_resolver<InternetProtocol, Executor>` are used to perform name resolution. Name resolution is the translation of a host name and service name into a sequence of endpoints, or the translation of an endpoint into its corresponding host name and service name.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace ip {

 template<class InternetProtocol, class Executor = executor>
 class basic_resolver : public resolver_base
 {
 public:
 // types:
```

```

using executor_type = io_context::executor_typeExecutor;
using protocol_type = InternetProtocol;
using endpoint_type = typename InternetProtocol::endpoint;
using results_type = basic_resolver_results<InternetProtocol>;

template<class OtherExecutor>
using rebind_executor =
 basic_resolver<InternetProtocol, OtherExecutor>;

// [internet.resolver.cons], construct / copy / destroy:

explicit basic_resolver(io_context& ctxconst executor_type& ex);
template<class ExecutionContext> explicit basic_resolver(ExecutionContext& ctx);
basic_resolver(const basic_resolver&) = delete;
basic_resolver(basic_resolver&& rhs) noexcept;

```

Update the `basic_resolver` constructors [internet.resolver.cons] as follows:

## 21.17.1 `ip::basic_resolver` constructors [internet.resolver.cons]

- ```

explicit basic_resolver(io_context& ctxconst executor_type& ex);

```
- 1 *Postconditions:* `get_executor()` == ~~`ctx.get_executor()`~~`ex`.
- ```

template<class ExecutionContext> explicit basic_resolver(ExecutionContext& ctx);

 Postconditions: get_executor() == ctx.get_executor().
 Remarks: This function shall not participate in overload resolution unless is -
convertible<ExecutionContext&, execution_context&>::value is true and is -
constructible<executor_type, typename ExecutionContext::executor_type>::value is true.

```
- ```

basic_resolver(basic_resolver&& rhs) noexcept;

```
- 2 *Effects:* Move constructs an object of class `basic_resolver<InternetProtocol>` that refers to the state originally represented by `rhs`.
 - 3 *Postconditions:* `get_executor()` == `rhs.get_executor()`.

Update the `ip::tcp` synopsis [internet.tcp] as follows:

21.19 Class `ip::tcp` [internet.tcp]

- 1 The class `tcp` encapsulates the types and flags necessary for TCP sockets.

```

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace ip {

class tcp
{
public:
    // types:
    using endpoint = basic_endpoint<tcp>;
    using resolver = basic_resolver<tcp>;
    using socket = basic_stream_socket<tcp>;
    template<class Executor>
        using socket_for = basic_stream_socket<tcp, Executor>;
    using acceptor = basic_socket_acceptor<tcp>;
    using iostream = basic_socket_iostream<tcp>;
    class no_delay;

    // static members:
    static constexpr tcp v4() noexcept;
    static constexpr tcp v6() noexcept;

    tcp() = delete;
};

```

```

// [internet.tcp.comparisons], tcp comparisons:
constexpr bool operator==(const tcp& a, const tcp& b) noexcept;
constexpr bool operator!=(const tcp& a, const tcp& b) noexcept;

} // namespace ip
} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

Update the `ip::udp` synopsis [internet.udp] as follows:

21.20 Class `ip::udp`

[internet.udp]

- 1 The class `udp` encapsulates the types and flags necessary for UDP sockets.

```

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace ip {

class udp
{
public:
// types:
using endpoint = basic_endpoint<udp>;
using resolver = basic_resolver<udp>;
using socket = basic_datagram_socket<udp>;
template<class Executor>
using socket_for = basic_datagram_socket<udp, Executor>;

// static members:
static constexpr udp v4() noexcept;
static constexpr udp v6() noexcept;

udp() = delete;
};

// [internet.udp.comparisons], udp comparisons:
constexpr bool operator==(const udp& a, const udp& b) noexcept;
constexpr bool operator!=(const udp& a, const udp& b) noexcept;

} // namespace ip
} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

7. History

Changes in R3:

Note: changed sections in R3 are indicated by a pale blue background with a dark blue line on the right margin.

- Added `socket_for` to `AcceptableProtocol` requirements, class `ip::tcp`, and class `ip::udp`.
- Renamed template type alias `basic_socket_acceptor::rebind_socket_executor` to `socket_type_for`.

Changes in R2:

- Changed `rebind_executor` to be a template type alias.
- Reinstated `basic_socket_acceptor::socket_type`.
- Added template type alias `basic_socket_acceptor::rebind_socket_executor`.
- Changed `basic_socket_acceptor::accept` and `basic_socket_acceptor::async_accept` overloads that take an executor (or execution context) to use the new `rebind_socket_executor` type alias.

Changes in R1:

- Added detailed wording.

- Added discussion of specialization of I/O object types.
- Updated implementation experience.